Autonomous Frontier Exploration — Setup & Run Guide

Team AirMouse · NIDAR 2.0 Mission 2 · Written 24 Aug 2026 Validated on: Ubuntu 22.04 · ROS 2 Humble · Gazebo Harmonic · ArduPilot SITL (iris, 2D lidar) · slam_toolbox · 8GB RAM VM (VMware Fusion, software rendering)

This document takes a working drone simulation (Gazebo + ArduPilot SITL + MAVProxy + slam_toolbox producing `/map` in RViz) and adds fully autonomous maze exploration on top of it, using only unmodified upstream packages plus two parameter files and a 30-line message adapter.

---

1. Architecture — read this before running anything

```
/scan (2D lidar) ──> slam_toolbox ──> /map (occupancy grid) + TF: map→odom
                                          │
                     ┌────────────────────┘
                     ▼
     frontier_exploration_ros2          ← decides WHERE to go
       reads /map + both Nav2 costmaps,
       finds frontiers (free/unknown boundaries),
       scores & orders them (MRTSP + bounded-horizon DP),
       sends ONE pose at a time as a NavigateToPose ACTION
                     │
                     ▼
     Nav2                               ← decides HOW to get there
       global planner: A*/Dijkstra path on the global costmap
       DWB controller: follows the path → velocity commands
       publishes geometry_msgs/Twist on /cmd_vel
                     │
                     ▼
     cmd_vel_stamper.py                 ← message-type adapter (packaging only)
       Twist → TwistStamped (adds timestamp + frame_id: base_link)
                     │
                     ▼
     /ap/v1/cmd_vel ──> ArduPilot AP_DDS ← does the FLYING
       ArduPilot's own controllers handle attitude, altitude hold, motors
```

Division of responsibility, one line: the explorer decides *where*, Nav2 decides *how*, ArduPilot *flies it*. There is no custom planning, control, or exploration logic anywhere in this stack — three upstream packages, two config files, one adapter.

Why the Python adapter exists: Nav2 Humble publishes `geometry_msgs/Twist`. ArduPilot's DDS interface expects `geometry_msgs/TwistStamped` — identical velocity fields plus a header (timestamp + frame). ROS 2 type-checks strictly, so without conversion the connection silently never forms. The script copies the numbers unchanged and adds the header. It is packaging, not control. (Later ROS 2 distros moved Nav2 to TwistStamped natively; we straddle that version boundary.)

Why it's an ACTION, not a topic: `NavigateToPose` is a ROS 2 action — goal → streamed feedback → final result (SUCCEEDED/ABORTED), cancellable. The explorer *requires* this interface: it uses the verdict to decide whether to advance to the next frontier or retry/suppress. A fire-and-forget topic could not close this loop. The explorer also reads both Nav2 costmaps to reject frontiers whose goal points are blocked — so Nav2 must be running even for frontier *selection*.

Known design gap (important): Nav2's `NavigateToPose` has **no built-in timeout**. A stuck goal hangs indefinitely. The explorer's suppression timeouts are the only bound on a stuck goal — they must stay enabled (see §6).

---

2. Prerequisites

Assumed already working (the team's standard daily sim procedure):

1. `ros2 launch ardupilot_gz_bringup iris_maze.launch.py rviz:=true use_gz_tf:=true lidar_dim:=2`
2. MAVProxy GCS: `mavproxy.py --console --map --master=:14550`
3. slam_toolbox: `ros2 launch slam_toolbox online_async_launch.py slam_params_file:=$HOME/mapper_params.yaml use_sim_time:=true`
4. RViz shows the map growing when the drone moves.

Hard requirements this guide depends on:

- **TF chain `map → odom → base_link` must resolve** (comes from `use_gz_tf:=true` + slam_toolbox). Verify: `ros2 run tf2_ros tf2_echo map base_link` prints transforms. If it prints "frame does not exist", your sim stack is not (fully) running — this is not a config problem.
- Topics `/scan`, `/map`, `/odometry` (nav_msgs/Odometry), `/ap/v1/cmd_vel` exist.
- ~2.5 GB free disk for Nav2 and the workspace. Check `df -h` first; below ~2 GB free, clean up before installing.

---

3. Installation

3.1 Nav2

```
sudo apt update
sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup ros-humble-diagnostic-updater
```

`ros-humble-diagnostic-updater` is listed explicitly because on one machine it was missing (likely removed by an earlier `apt autoremove`) and Nav2's lifecycle manager died on startup with `libdiagnostic_updater.so: cannot open shared object file` — see §7, error 2.

Do NOT install `ros-humble-turtlebot3-gazebo` on a machine running Gazebo Harmonic. It pulls Gazebo Classic (gazebo11), and both ship a binary named `gz`, so the packages hard-conflict — apt will try to remove `gz-tools2`, which breaks the `gz sim` command your drone sim launches through. (§7, error 1.)

3.2 The explorer package

Built from source, unmodified, in its own workspace:

```
mkdir -p ~/explore_ws/src
cd ~/explore_ws/src
git clone https://github.com/mertgulerx/frontier_exploration_ros2.git
cd ~/explore_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build --packages-select frontier_exploration_ros2
```

Humble support exists since v1.6.1 (main branch is fine). If the build dies silently on a low-RAM VM (`c++: fatal error: Killed`), rerun with `colcon build --packages-select frontier_exploration_ros2 --executor sequential` and close Gazebo during the build.

3.3 Config files

Create `~/nidar_config/` and place the three files from §4 in it. Every launch command below points at this directory.

---

4. The three files

Full contents follow. Copy them exactly — YAML type rules apply (see §7, error 4: a float parameter written as `1` instead of `1.0` crashes the node at startup).

4.1 `~/nidar_config/nav2_params_iris.yaml`

Full Nav2 Humble configuration for the iris treated as a 2D robot at fixed altitude. No amcl, no map_server — slam_toolbox owns `/map` and the `map→odom` transform. Launched via `nav2_bringup navigation_launch.py`.

```
# Nav2 params for iris quadcopter treated as a 2D robot at fixed altitude.
# ROS 2 Humble, nav2_bringup navigation_launch.py (no amcl, no map_server —
# slam_toolbox owns /map and map->odom).
#
# VERIFIED against live TF tree + topic list on 2026-08-24:
#   map -> odom -> base_link confirmed; /odometry is nav_msgs/Odometry.
# Original checklist kept for reference:
#   1. Frame names: ros2 run tf2_ros tf2_echo map base_link
#      and: ros2 run tf2_ros tf2_echo odom base_link
#      If your frames differ (e.g. base_link_frd), fix every robot_base_frame
#      and global_frame below.
#   2. Odom topic: ros2 topic list | grep local_position
#   3. Scan topic: assumed /scan.

bt_navigator:
  ros__parameters:
    use_sim_time: true
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odometry
    bt_loop_duration: 10
    default_server_timeout: 20
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_compute_path_through_poses_action_bt_node
      - nav2_smooth_path_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_assisted_teleop_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_drive_on_heading_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_is_stuck_condition_bt_node
      - nav2_goal_reached_condition_bt_node
      - nav2_goal_updated_condition_bt_node
      - nav2_globally_updated_goal_condition_bt_node
      - nav2_is_path_valid_condition_bt_node
      - nav2_initial_pose_received_condition_bt_node
```

```
      - nav2_reinitialize_global_localization_service_bt_node
      - nav2_rate_controller_bt_node
      - nav2_distance_controller_bt_node
      - nav2_speed_controller_bt_node
      - nav2_truncate_path_action_bt_node
      - nav2_truncate_path_local_action_bt_node
      - nav2_goal_updater_node_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node
      - nav2_round_robin_node_bt_node
      - nav2_transform_available_condition_bt_node
      - nav2_time_expired_condition_bt_node
      - nav2_path_expiring_timer_condition
      - nav2_distance_traveled_condition_bt_node
      - nav2_single_trigger_bt_node
      - nav2_goal_updated_controller_bt_node
      - nav2_is_battery_low_condition_bt_node
      - nav2_navigate_through_poses_action_bt_node
      - nav2_navigate_to_pose_action_bt_node
      - nav2_remove_passed_goals_action_bt_node
      - nav2_planner_selector_bt_node
      - nav2_controller_selector_bt_node
      - nav2_goal_checker_selector_bt_node
      - nav2_controller_cancel_bt_node
      - nav2_path_longer_on_approach_bt_node
      - nav2_wait_cancel_bt_node
      - nav2_spin_cancel_bt_node
      - nav2_back_up_cancel_bt_node
      - nav2_assisted_teleop_cancel_bt_node
      - nav2_drive_on_heading_cancel_bt_node

controller_server:
  ros__parameters:
    use_sim_time: true
    # 10 Hz instead of 20 — VM CPU budget.
    controller_frequency: 10.0
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
    min_theta_velocity_threshold: 0.001
    failure_tolerance: 0.5
    progress_checker_plugin: "progress_checker"
    goal_checker_plugins: ["general_goal_checker"]
    controller_plugins: ["FollowPath"]

    progress_checker:
      plugin: "nav2_controller::SimpleProgressChecker"
      required_movement_radius: 0.3
      movement_time_allowance: 15.0

    general_goal_checker:
      stateful: True
      plugin: "nav2_controller::SimpleGoalChecker"
      xy_goal_tolerance: 0.60
      # 360-degree lidar: final heading is irrelevant, don't spin at goals.
      yaw_goal_tolerance: 6.28

    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      debug_trajectory_details: False
      min_vel_x: 0.0
      min_vel_y: 0.0
      # Conservative for 1m corridors; raise after it works.
      max_vel_x: 0.5
      max_vel_y: 0.0
      max_vel_theta: 1.0
      min_speed_xy: 0.0
      max_speed_xy: 0.5
      min_speed_theta: 0.0
      acc_lim_x: 0.8
      acc_lim_y: 0.0
      acc_lim_theta: 1.5
      decel_lim_x: -0.8
      decel_lim_y: 0.0
      decel_lim_theta: -1.5
      vx_samples: 20
      vy_samples: 1
      vtheta_samples: 20
      sim_time: 1.7
      linear_granularity: 0.05
      angular_granularity: 0.025
      # Generous: VM + SITL TF latency.
      transform_tolerance: 0.5
      xy_goal_tolerance: 0.60
      trans_stopped_velocity: 0.25
      short_circuit_trajectory_evaluation: True
      stateful: True
      critics: ["Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
```

```
      BaseObstacle.scale: 0.02
      PathAlign.scale: 32.0
      PathAlign.forward_point_distance: 0.1
      GoalAlign.scale: 24.0
      GoalAlign.forward_point_distance: 0.1
      PathDist.scale: 32.0
      GoalDist.scale: 24.0

local_costmap:
  local_costmap:
    ros__parameters:
      use_sim_time: true
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 4
      height: 4
      resolution: 0.05
      # Iris incl. props ~0.30. THE critical number for 1m corridors:
      # free centerline band = corridor_width - 2*robot_radius.
      robot_radius: 0.30
      plugins: ["obstacle_layer", "inflation_layer"]
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: True
          marking: True
          data_type: "LaserScan"
          raytrace_max_range: 8.0
          raytrace_min_range: 0.0
          obstacle_max_range: 6.0
          obstacle_min_range: 0.0
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        # High scaling = cost decays fast from walls, so the 0.4m-wide
        # non-lethal band in a 1m corridor stays cheap enough to plan through.
        cost_scaling_factor: 5.0
        inflation_radius: 0.35
      always_send_full_costmap: True

global_costmap:
  global_costmap:
    ros__parameters:
      use_sim_time: true
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      robot_radius: 0.30
      resolution: 0.05
      # Critical for exploration: unknown must NOT be treated as free,
      # or frontier semantics degrade and the planner cuts through unmapped space.
      track_unknown_space: true
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        # slam_toolbox publishes transient_local.
        map_subscribe_transient_local: True
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: True
          marking: True
          data_type: "LaserScan"
          raytrace_max_range: 8.0
          raytrace_min_range: 0.0
          obstacle_max_range: 6.0
          obstacle_min_range: 0.0
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 5.0
        inflation_radius: 0.35
      always_send_full_costmap: True

planner_server:
  ros__parameters:
    use_sim_time: true
```

```
    expected_planner_frequency: 5.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      # Frontier goals sit at the known/unknown boundary; the planner
      # must be allowed to plan near unknown space.
      allow_unknown: true

smoother_server:
  ros__parameters:
    use_sim_time: true
    smoother_plugins: ["simple_smoother"]
    simple_smoother:
      plugin: "nav2_smoother::SimpleSmoother"
      tolerance: 1.0e-10
      max_its: 1000
      do_refinement: True

behavior_server:
  ros__parameters:
    use_sim_time: true
    costmap_topic: local_costmap/costmap_raw
    footprint_topic: local_costmap/published_footprint
    cycle_frequency: 10.0
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]
    spin:
      plugin: "nav2_behaviors/Spin"
    backup:
      plugin: "nav2_behaviors/BackUp"
    drive_on_heading:
      plugin: "nav2_behaviors/DriveOnHeading"
    wait:
      plugin: "nav2_behaviors/Wait"
    global_frame: odom
    robot_base_frame: base_link
    transform_tolerance: 0.5
    simulate_ahead_time: 2.0
    max_rotational_vel: 1.0
    min_rotational_vel: 0.4
    rotational_acc_lim: 3.2

waypoint_follower:
  ros__parameters:
    use_sim_time: true
    loop_rate: 20
    stop_on_failure: false
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: True
      waypoint_pause_duration: 200

velocity_smoother:
  ros__parameters:
    use_sim_time: true
    smoothing_frequency: 20.0
    scale_velocities: False
    feedback: "OPEN_LOOP"
    max_velocity: [0.5, 0.0, 1.0]
    # Small negative x allowed so the BackUp recovery can work.
    min_velocity: [-0.25, 0.0, -1.0]
    max_accel: [0.8, 0.0, 1.5]
    max_decel: [-0.8, 0.0, -1.5]
    odom_topic: "/odometry"
    odom_duration: 0.1
    deadband_velocity: [0.0, 0.0, 0.0]
    velocity_timeout: 1.0
```

Key choices in this file (and why):

Setting	Value	Rationale
`robot_base_frame`	`base_link` (everywhere)	The drone's actual base frame; ground-robot default is `base_footprint`, which does not exist here
`odom_topic`	`/odometry` (2 places)	Ground-truth odometry from the Gazebo bridge; there is no MAVROS in this stack
`robot_radius`	0.30	Iris incl. props. In a 1 m corridor the free centreline band = 1 − 2×radius = 0.4 m
`inflation_radius`	0.35 (was 0.45)	0.45 costed corridors nearly shut; goals near walls became unreachable (§7, error 9)
`xy_goal_tolerance`	0.60 (2 places; was 0.25)	Frontiers only need to be *observed*, not touched; the lidar sees metres. 0.25 caused permanent "last 20 cm" stalls
`yaw_goal_tolerance`	6.28	360° lidar → final heading is meaningless; never spin at goals
`critics`	RotateToGoal **removed**	It commanded pure spins at goals (measured `angular.z: 1.0`, 11 recoveries) optimizing a heading we declared irrelevant (§7, error 8)
`track_unknown_space`	true (global costmap)	Mandatory for exploration: unknown ≠ free, or the planner routes through unmapped space and frontier semantics break
`allow_unknown`	true (planner)	Frontier goals sit at the known/unknown boundary; the planner must be allowed to plan near unknown
`controller_frequency`	10 Hz (default 20)	VM CPU budget
`transform_tolerance`	0.5	SITL + VM TF latency
`max_vel_x`	0.5	Conservative for 1 m corridors; raise once stable

4.2 `~/nidar_config/frontier_params_iris.yaml`

Explorer configuration. Based on the package's shipped `config/params.yaml` (parameter names verified against the repo, not guessed), with drone/arena tuning. Values below reflect the state **after** run-1 and run-2 fixes.

```
frontier_explorer:
  ros__parameters:

    # --------------- TOPICS --------------- #
    map_topic: /map
    costmap_topic: /global_costmap/costmap
    local_costmap_topic: /local_costmap/costmap
    navigate_to_pose_action_name: navigate_to_pose
    global_frame: map

    # CHANGED (was base_footprint): drone has no footprint frame.
    # Verify actual frame name with: ros2 run tf2_ros tf2_echo map base_link
    robot_base_frame: base_link

    frontier_marker_topic: /explore/frontiers
    selected_frontier_topic: /explore/selected_frontier
    optimized_map_topic: /explore/optimized_map
    completion_event_topic: exploration_complete
    frontier_marker_scale: 0.15

    # --------------- QoS --------------- #
    # slam_toolbox publishes /map transient_local — packaged default is correct.
    map_qos_durability: transient_local
    map_qos_reliability: reliable
    map_qos_depth: 1
    map_qos_autodetect_on_startup: false
    map_qos_autodetect_timeout_s: 5.0
    costmap_qos_reliability: reliable
    costmap_qos_depth: 10
    local_costmap_qos_reliability: inherit
    local_costmap_qos_depth: -1

    # --------------- GENERAL --------------- #
    map_processing_rate_hz: 1.0
    autostart: true

    # CHANGED (was false): keep the control service so you can
    # frontier_exploration_ctl stop / start during testing.
    control_service_enabled: true

    completion_event_enabled: true
    escape_enabled: true

    # Set true later: this maps directly onto the competition's
    # exit-through-entry-point requirement. False for first runs.
    return_to_start_on_complete: false

    mrtsp_solver: dp
    dp_solver_candidate_limit: 12

    # CHANGED (was 8): 15x15m maze rarely has deep useful lookahead;
    # short horizon replans better in corridors and saves CPU on the VM.
```

```
    dp_planning_horizon: 4

    frontier_map_optimization_enabled: true
    goal_preemption_enabled: true
    goal_preemption_min_interval_s: 2.0
    goal_skip_on_blocked_goal: true

    # CHANGED (was false): your SLAM+lidar update loop on the VM is slow;
    # a small settle after each goal avoids re-goaling on a stale map.
    post_goal_settle_enabled: true
    post_goal_min_settle: 1.50

    # Suppression ON (learned the hard way in run 1: without it, an
    # unreachable frontier is re-dispatched forever).
    frontier_suppression_enabled: true
    all_frontiers_suppressed_behavior: return_to_start

    # --------------- MRTSP Scoring --------------- #
    weight_distance_wd: 2.0
    weight_gain_ws: 1.0

    # CHANGED (was 0.50): match the linear speed cap you fly in GUIDED.
    # Update if you cap slower for the maze.
    max_linear_speed_vmax: 0.8

    max_angular_speed_wmax: 1.0

    # CHANGED (was 1.5): maze sightlines cap around 4-6m at junctions;
    # 1.5 is TB3-lidar tuning. This is a scoring value, not a sensor spec.
    sensor_effective_range_m: 4.0

    # --------------- Navigation Safety --------------- #
    # Recommended 2.5x robot radius; iris radius ~0.30 -> 0.75, keep 0.8.
    frontier_selection_min_distance: 0.8
    frontier_visit_tolerance: 0.40

    # --------------- Map Optimization --------------- #
    frontier_candidate_min_goal_distance_m: 1.0
    occ_threshold: 65

    # CHANGED (was 5): at 0.05m resolution, 8 cells = 40cm — kills noise
    # frontiers that are too small to be a real 1m-grid corridor opening.
    min_frontier_size_cells: 8

    sigma_s: 2.0
    sigma_r: 30.0
    dilation_kernel_radius_cells: 1

    # --------------- Preemption --------------- #
    goal_preemption_complete_if_within_m: 0.50

    # RPLidar A1 spec range; matches your sim lidar config.
    goal_preemption_lidar_range_m: 12.0
    goal_preemption_lidar_fov_deg: 360.0
    goal_preemption_lidar_ray_step_deg: 1.0
    goal_preemption_lidar_min_reveal_length_m: 0.5
    goal_preemption_lidar_yaw_offset_deg: 0.0

    # --------------- Suppression (inactive until enabled above) --------------- #
    frontier_suppression_attempt_threshold: 2
    frontier_suppression_base_size_m: 1.0
    frontier_suppression_expansion_size_m: 0.5
    frontier_suppression_timeout_s: 60.0
    frontier_suppression_no_progress_timeout_s: 25.0
    frontier_suppression_progress_epsilon_m: 0.25

    # CHANGED (was 15): SITL + MAVROS + Nav2 bring-up on the VM is slow.
    frontier_suppression_startup_grace_period_s: 30.0

    frontier_suppression_max_attempt_records: 256
    frontier_suppression_max_regions: 64
```

Key choices in this file (and why):

Setting	Value	Rationale
`robot_base_frame`	`base_link`	Packaged default is `base_footprint` (TB3); drone has no footprint frame
`frontier_suppression_enabled`	**true**	THE anti-loop mechanism. Run 1 had it off: one unreachable frontier was re-dispatched ~30 times in 15 min (§7, error 6)
`frontier_suppression_timeout_s`	60.0	Hard cap per goal attempt — the only timeout in the whole chain, since Nav2 has none
`frontier_suppression_no_progress_timeout_s`	25.0	Cancels goals whose map reveal has stalled
`frontier_suppression_startup_grace_period_s`	30.0	Nav2 + SITL bring-up is slow on the VM; don't poison suppression memory during startup
`weight_distance_wd`	2.0 (was 1.0)	Biases selection toward nearer frontiers → less cross-arena backtracking. Relevant to the <15 min speed bonus
`dp_planning_horizon`	4 (packaged: 8)	15×15 m maze rarely rewards deep lookahead; short horizons replan better in corridors, cheaper on the VM
`sensor_effective_range_m`	4.0 (packaged: 1.5)	Scoring value for expected information gain — maze sightlines cap around 4–6 m at junctions. NOT the lidar spec
`max_linear_speed_vmax`	0.8	Must roughly match the speed the drone actually flies, or MRTSP's time-cost estimates skew
`min_frontier_size_cells`	8 (packaged: 5)	At 0.05 m resolution, 8 cells = 40 cm — kills noise frontiers too small to be a real 1 m corridor opening
`frontier_candidate_min_goal_distance_m`	1.0	Pushes dispatch points further off walls (mitigates the inflation squeeze). Recommended ≥2.5× robot radius
`control_service_enabled`	true	Enables `frontier_exploration_ctl stop/start` — essential while testing
`return_to_start_on_complete`	false (for now)	Flip to true for competition behaviour: maps directly onto the exit-through-entry-point requirement
`map_qos_durability`	transient_local	Matches slam_toolbox's `/map` QoS; wrong durability = explorer never receives the map

4.3 `~/nidar_config/cmd_vel_stamper.py`

The message-type adapter. Run directly with `python3` — no package, no build.

```
#!/usr/bin/env python3
"""Bridge: Nav2 /cmd_vel (Twist) -> ArduPilot AP_DDS /ap/v1/cmd_vel (TwistStamped).

Nav2 Humble publishes unstamped Twist in the base frame; AP_DDS expects
TwistStamped with frame_id set — same convention the validated keyboard
teleop uses (stamped:=true, frame_id:=base_link).

Run directly, no package needed:
    python3 cmd_vel_stamper.py
Override frame if the TF tree says otherwise:
    python3 cmd_vel_stamper.py --ros-args -p frame_id:=<your_base_frame>
"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist, TwistStamped


class CmdVelStamper(Node):
    def __init__(self):
        super().__init__('cmd_vel_stamper')
        self.declare_parameter('frame_id', 'base_link')
        self.declare_parameter('in_topic', '/cmd_vel')
        self.declare_parameter('out_topic', '/ap/v1/cmd_vel')
        frame = self.get_parameter('frame_id').value
        self._frame = frame
        self._pub = self.create_publisher(
            TwistStamped, self.get_parameter('out_topic').value, 10)
        self._sub = self.create_subscription(
            Twist, self.get_parameter('in_topic').value, self._cb, 10)
        self.get_logger().info(
            f"Bridging {self.get_parameter('in_topic').value} (Twist) -> "
            f"{self.get_parameter('out_topic').value} (TwistStamped, frame '{frame}')")

    def _cb(self, msg: Twist):
        out = TwistStamped()
        out.header.stamp = self.get_clock().now().to_msg()
        out.header.frame_id = self._frame
        out.twist = msg
        self._pub.publish(out)


def main():
    rclpy.init()
    node = CmdVelStamper()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass


if __name__ == '__main__':
    main()
```

Line-by-line: it declares three parameters (frame, in-topic, out-topic), subscribes to `/cmd_vel` (`Twist`), and on every message constructs a `TwistStamped` whose header carries the current clock time and `frame_id: base_link`, with the velocity fields copied unchanged, publishing to `/ap/v1/cmd_vel`. The `frame_id: base_link` matters: it tells ArduPilot the velocities are body-relative (forward = nose direction), which is exactly Nav2's convention — the same convention the team's keyboard-teleop command already validated on this topic.

---

5. Run procedure

Seven terminals total. **Source in every terminal:**

```
source /opt/ros/humble/setup.bash
source ~/explore_ws/install/setup.bash     # needed in T7 (and harmless elsewhere)
```

The ordering below is a verification ladder — each step validates exactly one layer, so when something breaks you know where it is. **Do not skip steps on a first run.**

T1–T4 — the standard sim stack (unchanged)

```
# T1: simulation (2D lidar + TF are mandatory arguments)
ros2 launch ardupilot_gz_bringup iris_maze.launch.py rviz:=true use_gz_tf:=true lidar_dim:=2

# T2: GCS
mavproxy.py --console --map --master=:14550

# T3: SLAM
ros2 launch slam_toolbox online_async_launch.py slam_params_file:=$HOME/mapper_params.yaml use_sim_time:=true

# (T4 was teleop in the old procedure — NOT needed for autonomous exploration.
#  Keep the command handy as a debug probe: if the exploration chain ever does
#  nothing, two seconds of teleop instantly tells you whether the fault is
#  above or below /ap/v1/cmd_vel.)
```

Wait until RViz shows the map and Global Status: Ok.

Verify interfaces (30 seconds, do not skip)

```
ros2 run tf2_ros tf2_echo map base_link        # must print transforms
ros2 topic list | grep -E "^/scan$|^/map$|^/odometry$|/ap/v1/cmd_vel"
ros2 topic info /odometry                      # Type: nav_msgs/msg/Odometry
```

T5 — Nav2

```
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true \
  params_file:=$HOME/nidar_config/nav2_params_iris.yaml
```

Give it ~15 s for lifecycle activation. Then verify: `ros2 topic hz /global_costmap/costmap` shows messages (the measured rate will read far below the configured 1 Hz on a slow VM — that's real-time-factor distortion, not a fault).

Costmap sanity check (the single most important visual check): in RViz add Map displays for `/global_costmap/costmap` and `/local_costmap/costmap`, set Color Scheme to `costmap`. Look down a mapped corridor: you must see a passable (non-lethal) band along the centreline between the inflated wall bands. If corridors are costed shut wall-to-wall, exploration is dead before it starts — lower `inflation_radius` and relaunch Nav2 before proceeding.

T2 (MAVProxy) — takeoff

```
param set ARMING_CHECK 0     # simulation only
mode guided
arm throttle
takeoff 1
```

Confirm the iris is actually hovering in Gazebo before continuing (ArduPilot silently ignores velocity commands unless armed, in GUIDED, and airborne).

T6 — the bridge

```
python3 ~/nidar_config/cmd_vel_stamper.py
```

Verify: `ros2 topic info /ap/v1/cmd_vel` → Publisher count ≥ 1, Subscription count 1.

Manual goal test (validates the whole chain except the explorer)

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
"{pose: {header: {frame_id: map}, pose: {position: {x: 2.0, y: 0.0, z: 0.0}, orientation: {w: 1.0}}}}" --feedback
```

Pick x/y a couple of metres away **inside already-mapped free space** (check current position with `ros2 topic echo /odometry --once`). Expected: `distance_remaining` streams down, the drone flies in Gazebo, final line `Goal finished with status: SUCCEEDED`. Note `number_of_recoveries` in the feedback — it should be low single digits; a high number means Nav2 is fighting the corridors (see §8).

Alternative if RViz is responsive: toolbar → **2D Goal Pose** → click a mapped point. (First set Global Options → Fixed Frame to `map`, otherwise the click is interpreted in the wrong frame.)

T7 — the explorer

```
ros2 launch frontier_exploration_ros2 frontier_explorer.launch.py use_sim_time:=true \
  params_file:=$HOME/nidar_config/frontier_params_iris.yaml
```

Autostart is on. Within seconds of the first map callback you should see `Sending frontier goal (mrtsp): frontier reference=(x, y) ... size=N` followed by `Frontier goal accepted`, and the drone starts exploring on its own. Watch the *terminal*, not RViz — at 2–3 fps on the VM, the log is the truth.

Reading the log: `size` is the frontier's cell count. The same frontier reappearing with **growing** size = the drone is revealing it (working,

slowly). Same frontier with **frozen** size on repeat = it can't get close enough (suppression should intervene after 2 failed attempts / the timeouts). `STATUS_ABORTED` followed by a *different* frontier = suppression doing its job.

Controls, completion, saving the artifact

```
frontier_exploration_ctl stop        # pause exploration (any sourced terminal)
frontier_exploration_ctl start       # resume
```

On exhausting all frontiers the explorer publishes on `exploration_complete` and stops. Save the map immediately — this is the 220-point deliverable:

```
ros2 run nav2_map_server map_saver_cli -f ~/nidar_config/maze_run1
```

VM performance notes

- RViz is optional — the pipeline runs headless. If RViz hangs (unclickable at low fps), `pkill rviz2` is safe: it's a pure subscriber, nothing else is affected.
- In RViz keep only Map + Global CostMap + TF; PointCloud2 and LaserScan displays are the main render cost.
- Restarting order matters: slam_toolbox restart = fresh map (exploration starts over). Nav2 restart = safe mid-run. Explorer restart = safe (re-reads the map).

---

6. Errors encountered, in the order we hit them

1. Gazebo Classic conflicts with Gazebo Harmonic (blocked install). Symptom: `gz-tools2 : Conflicts: gazebo (>= 11.0.0)` when installing turtlebot3-gazebo. Cause: gazebo11 and gz-tools2 both ship a binary named `gz`; the packaging declares a hard conflict. Consequence: no TurtleBot3 smoke test on a Harmonic machine — we validated directly on the drone stack using the incremental ladder in §5 instead. Lesson: never let apt remove `gz-tools2`.

2. Nav2 lifecycle manager dies instantly. Symptom: `lifecycle_manager: error while loading shared libraries: libdiagnostic_updater.so`, then every Nav2 node stuck at "Waiting on external lifecycle transitions" forever and no costmap topics. Cause: `ros-humble-diagnostic-updater` missing (probably a casualty of an earlier `apt autoremove --purge` during disk cleanup). Fix: `sudo apt install ros-humble-diagnostic-updater`, relaunch. General rule: if every Nav2 node waits forever, the lifecycle manager died — read the first error line of the launch output, not the last.

3. "frame does not exist" on tf2_echo. Symptom: `Invalid frame ID "map" ... frame does not exist`. Cause: the sim stack wasn't running — TF only exists while its publishers are alive. Not a naming problem. Rule: all TF/topic verification must be done against the live stack.

4. Explorer crashes on startup with InvalidParameterTypeException. Symptom: `parameter 'frontier_candidate_min_goal_distance_m' has invalid type ... {double}, setting it to {integer} is not allowed`. Cause: a float parameter edited to `1` instead of `1.0`. ROS 2 YAML parsing is strictly typed. Rule: every float parameter keeps an explicit decimal point.

5. RViz freezes / becomes unclickable (2–6 fps). Cause: not RAM (3.8 GB was free) — CPU/render starvation: llvmpipe software rendering competing with Gazebo physics, SITL, and 8 Nav2 processes. Fix: kill RViz (safe — pure subscriber), relaunch with minimal displays, or run headless and use the explorer log + `ros2 topic echo` as instruments.

6. Explorer loops forever on one unreachable frontier (run 1's main failure). Symptom: the same frontier `(0.72, 6.34)`, same dispatch point, `size` frozen at 245, re-dispatched ~30 times over ~15 minutes; the drone hovering near a wall. Cause: `frontier_suppression_enabled: false` (we'd disabled it to reduce moving parts on the first run — a mistake). Suppression is the designed anti-loop mechanism: after N failed attempts or no map progress, it blacklists the region and moves on. Fix: enable it. It is now considered mandatory, not optional.

7. bt_navigator wedges after an aborted goal; all subsequent goals rejected. Symptom: one `STATUS_ABORTED`, then an endless stream of `Frontier goal was rejected`. Fix: restart Nav2 (Ctrl-C T5, relaunch). The explorer can stay up. Related design note: `NavigateToPose` has **no default timeout** — the aborted goal had been chased indefinitely first. The explorer's suppression timeouts are the only bound in the chain; for a 30-minute competition run this must be configured deliberately.

8. Drone spins in place at goals; recoveries pile up. Symptom (measured): `/cmd_vel` showing `angular.z: 1.0` with zero linear; `number_of_recoveries: 11`; `distance_remaining: 0.0` while the goal never succeeds. Cause: the DWB `RotateToGoal` critic — built for robots with a "front" — commanding heading alignment that a 360° lidar drone doesn't need, interacting badly with recovery Spin behaviors. Fix: remove `RotateToGoal` from the critics list (and delete its parameter lines), with `yaw_goal_tolerance: 6.28` making heading explicitly irrelevant.

9. Goals stall "20 cm short" near walls (the inflation/tolerance squeeze). Symptom (measured): drone at (7.23, 3.08), goal at (7.62, 3.50) — 0.57 m apart with tolerance 0.40; controller cannot generate a legal forward trajectory because the remaining gap lies inside the inflation band; stall → recoveries → eventual ABORT. This was the root cause behind errors 6–8's visible behaviour. Fixes applied: `xy_goal_tolerance` 0.25 → 0.60 (frontiers need only be *observed* — the lidar reveals them from metres away), `inflation_radius` 0.45 → 0.35, `frontier_candidate_min_goal_distance_m` 0.8 → 1.0 (dispatch points further off walls).

---

7. Current state and open items

Working: full autonomous exploration validated end-to-end. Run 2 (suppression on, wd=2.0): six distinct frontiers explored with no repeat-lock; suppression correctly rotates away from failed goals after 2 attempts; the distance-weighting change visibly reduced cross-arena backtracking.

Primary metric going forward: `number_of_recoveries` per goal (from `/navigate_to_pose/_action/feedback`) and the SUCCEEDED/ABORTED ratio. These decide the open architecture question below.

Open question — partial Nav2 replacement: DWB is a differential-drive-style controller driving a holonomic quadcopter (our config: `max_vel_y: 0.0`), while ArduPilot already contains WPNAV, a purpose-built path follower for this airframe. The candidate design keeps Nav2's global planner + costmaps (via the `ComputePathToPose` action, which returns a path without driving) and replaces only the controller with a ~150–200-line shim feeding cell-centre waypoints to ArduPilot — which also lets the shim own the missing goal timeout. Principal risk: frame divergence (ArduPilot's EKF-origin frame vs. the SLAM map frame under loop-closure corrections). Decision trigger: if recoveries and near-wall ABORTs persist after §6's tuning, build it; if tuning suffices, don't. To be decided from measurements by mid-September, documented in the first report.

Also queued: `return_to_start_on_complete: true` (maps onto the exit-through-entry rule), an explore_lite (`m-explore-ros2`) comparison run on the same maze for the report (greedy nearest-frontier vs. MRTSP: total time + path length), and — explicitly *not* behind any of this — the survivor-detection pipeline (240 pts, still architecturally underspecified). TAILEOF cat guide_head.md guide_tail.md > /mnt/user-data/outputs/frontier_exploration_setup_guide.md wc -l /mnt/user-data/outputs/frontier_exploration_setup_guide.md